Projekt Bisektion

Name: Michael Strohmayer

Klasse: 3BHWII

Fach: SWP

Schuljahr 2025/26

Aufgabe 5 – Bisektionsverfahren

Der Code implementiert das Bisektionsverfahren zur Berechnung von Nullstellen.

Zuerst gibt es eine Funktion, die aus einer Texteingabe eine mathematische Funktion erstellt.

```python
# Aufgabe 5: Nullstellenbestimmung mit Bisektions-Verfahren
def funktion_aus_text_erstellen(formel: str):
    """Erstellt eine Funktion aus einem gegebenen String.
    formel: Ein String, der eine mathematische Funktion beschreibt, z.B. "x**2 - 25"
    Rückgabe: Eine Funktion, die die gegebene Formel auswertet.
    """
    erlaubte_werte = {
        "x": 0,
        "math": math,
        "sin": math.sin,
        "cos": math.cos,
        "tan": math.tan,
        "sqrt": math.sqrt,
        "sinh": math.sinh,
        "cosh": math.cosh,
        "pi": math.pi,
        "e": math.e,
    }

    def f(x: float) -> float:
        erlaubte_werte["x"] = x
        return eval(formel, {"__builtins__": {}}, erlaubte_werte)

    return f
```

Der Benutzer kann zum Beispiel „x**2 - 25“ eingeben, und daraus wird eine Funktion erzeugt, die später berechnet werden kann.

Danach wird das Bisektionsverfahren umgesetzt.

```python
# Bisektions-Verfahren
def bisektion(f,a ,b , epsilon)-> float:
    """f: Funktion, deren Nullstelle gefunden werden soll
    a: Anfang des Intervalls
    b: Ende des Intervalls
    epsilon: gewünschte Genauigkeit
    Rückgabe: Nullstelle von f im Intervall [a, b] oder None, wenn kein Vorzeichenwechsel vorliegt
    """
    try:
        if f(a) * f(b) >= 0: # Kein Vorzeichenwechsel
            print("Kein Vorzeichenwechsel")
            return None

        while (b - a) / 2 > epsilon: # Solange die Genauigkeit nicht erreicht ist
            c = (a + b) / 2 # Mitte des Intervalls

            if f(a) * f(c) < 0: # Vorzeichenwechsel zwischen a und c
                b = c
            else:
                a = c

        return (a + b) / 2

    except:
        print("Fehler")
        return None
```

Dabei wird ein Intervall mit zwei Grenzen verwendet. In jeder Iteration wird die Mitte berechnet. Anschließend wird geprüft, in welcher Hälfte des Intervalls sich die Nullstelle befindet. Dieses Intervall wird dann weiter verkleinert.

Das Verfahren läuft so lange, bis die gewünschte Genauigkeit erreicht ist.

Zusätzlich enthält der Code:

- Fehlerüberprüfungen (z. B. falsches Intervall oder ungültige Genauigkeit)
- Ausgabe der Anzahl der Iterationen
- Ausgabe der gefundenen Nullstelle

Am Ende gibt es ein Menü, in dem man:

- entweder eine eigene Funktion eingeben kann
- oder vorgegebene Testfälle ausführen kann

```python
#Printer für Eigene Funktion oder Testfälle
def solver():
    print("=== Aufgabe 5 ===")
    print("1 - Eigene Funktion")
    print("2 - Testfälle")

    wahl = input("Auswahl: ")

    if wahl == "1": #Eingabe der Funktion und des Intervalls sowie der gewünschten Genauigkeit
        formel = input("Funktion: ") #Eingabe der Funktion als String
        a = float(input("a: ")) #Eingabe des Intervallanfangs
        b = float(input("b: ")) #Eingabe des Intervallendes
        epsilon = float(input("epsilon: ")) #Eingabe der gewünschten Genauigkeit

        f = funktion_aus_text_erstellen(formel) #Erstellung der Funktion aus dem eingegebenen String
        x = bisektion(f, a, b, epsilon) #Berechnung der Nullstelle mit dem Bisektions-Verfahren

        print("Nullstelle:", x)

    elif wahl == "2": #Testfälle für die Funktion x^2 - n, um die Quadratwurzel von n zu finden
        for n in [25, 81, 144]:
            f = funktion_aus_text_erstellen(f"x**2 - {n}")
            x = bisektion(f, 0, n, 0.00001)

            print("n =", n, "| numerisch:", x, "| exakt:", math.sqrt(n))

    else:
        print("Ungültige Auswahl.") #Ausgabe bei ungültiger Auswahl


if __name__ == "__main__":
    solver()
```

Aufgabe 6 – Regula Falsi

Der Code funktioniert ähnlich wie das Bisektionsverfahren, verwendet aber eine andere Methode zur Berechnung der nächsten Näherung.

```python
#Regula Falsi Verfahren
def regula_falsi(f, a, b, epsilon)-> float:
    while True:
        """f: Funktion, deren Nullstelle gefunden werden soll
    a: Anfang des Intervalls
    b: Ende des Intervalls
    epsilon: gewünschte Genauigkeit
    Rückgabe: Nullstelle von f im Intervall [a, b] oder None, wenn kein Vorzeichenwechsel vorliegt
    """
        c = b - f(b) * (b - a) / (f(b) - f(a))# Berechnung der neuen Näherung c

        if abs(f(c)) < epsilon:# Überprüfung der Genauigkeit
            return c

        if f(a) * f(c) < 0:#Vorzeichenwechsel zwischen a und c
            a = c
        else:
            b = c
```

Statt der Mitte wird eine neue Stelle berechnet, indem eine Gerade zwischen zwei Punkten gelegt wird. Diese Methode kann oft schneller zur Lösung führen.

Der Ablauf ist ähnlich:

- Intervall prüfen
- neue Näherung berechnen
- Intervall anpassen
- wiederholen

Auch hier gibt es:

- Fehlerbehandlung
- Iterationszählung
- Testfälle
- ein Menü zur Auswahl

Aufgabe 7 – Visualisierung

Dieser Code erweitert das Bisektionsverfahren um eine grafische Darstellung.

Während das Verfahren läuft, werden zwei Diagramme angezeigt:

- oben der Funktionsgraph

```python
#Bisektions-Verfahren mit Grafik
def bisektion_grafik(f, a, b, epsilon)-> float:
    """f: Funktion, deren Nullstelle gefunden werden soll
    a: Anfang des Intervalls
    b: Ende des Intervalls
    epsilon: gewünschte Genauigkeit
    Rückgabe: Nullstelle von f im Intervall [a, b] oder None, wenn kein Vorzeichenwechsel vorliegt
    """
    if f(a) * f(b) >= 0: # Kein Vorzeichenwechsel
        print("Kein Vorzeichenwechsel.")
        return None

    loesungen = [] #Liste zur Speicherung der Lösungen für die Grafik
    genauigkeiten = [] #Liste zur Speicherung der Genauigkeiten für die Grafik
    iterationen = [] #Liste zur Speicherung der Iterationen für die Grafik

    fig, (ax1, ax2, ax3) = plt.subplots(3, 1) #Erstellung von drei Subplots für die Funktion, die Genauigkeit und die Lösungen

    iteration = 0 # Zähler für die Iterationen
```

- unten die Entwicklung der Genauigkeit und der aktuellen Lösung

```python
    while (b - a) / 2 > epsilon: # Solange die Genauigkeit nicht erreicht ist
        c = (a + b) / 2 # Mitte des Intervalls

        iteration += 1
        iterationen.append(iteration)
        loesungen.append(c)
        genauigkeiten.append(abs(f(c)))

        x = [] #Liste zur Speicherung der x-Werte für die Grafik
        y = [] #Liste zur Speicherung der y-Werte für die Grafik

        i = a - 2 # Startpunkt für die x-Werte, etwas kleiner als a, um die Funktion vor dem Intervall zu zeigen
        while i <= b + 2: # Endpunkt für die x-Werte, etwas größer als b, um die Funktion nach dem Intervall zu zeigen
            x.append(i) #Hinzufügen des aktuellen x-Werts zur Liste
            y.append(f(i)) #Hinzufügen des Funktionswerts für den aktuellen x-Wert zur Liste
            i += 0.1

        ax1.clear() #Clearing der Subplots, um die neuen Werte zu plotten
        ax2.clear() #Clearing der Subplots, um die neuen Werte zu plotten
        ax3.clear() #Clearing der Subplots, um die neuen Werte zu plotten

        ax1.plot(x, y) #Grafik der Funktion f(x) über das Intervall [a-2, b+2]
        ax1.axhline(0) #Horizontale Linie bei y=0, um die Nullstelle besser sichtbar zu machen
        ax1.axvline(c) #Vertikale Linie bei x=c, um die aktuelle Näherung besser sichtbar zu machen
        ax1.set_title("Bisektion") #Titel für die Funktionengrafik

        ax2.plot(iterationen, genauigkeiten) #Grafik der Genauigkeit über die Iterationen
        ax2.set_title("Aktuelle Genauigkeit |f(c)|") #Titel für die Genauigkeitsgrafik

        ax3.plot(iterationen, loesungen) #Grafik der Lösungen über die Iterationen
        ax3.set_title("Aktuelle Lösung c") #Titel für die Lösungsgrafik

        plt.tight_layout() #Anpassung des Layouts, damit die Subplots nicht überlappen
        plt.pause(0.5) #Pause, um die Grafik zu aktualisieren und die Iterationen sichtbar zu machen

        if f(a) * f(c) < 0: # Vorzeichenwechsel zwischen a und c
            b = c
        else:
            a = c

    plt.show()
    return (a + b) / 2 # Rückgabe der gefundenen Nullstelle
```

Nach jeder Iteration wird die Grafik aktualisiert, sodass eine Art Animation entsteht.

Zusätzlich kann man:

- eine eigene Funktion eingeben
- oder einen Testfall anzeigen lassen

Der Code zeigt dadurch sehr gut, wie sich die Lösung Schritt für Schritt verbessert.

```python
# Printer für die Aufgabe 7
def plotter():
    print("=== Aufgabe 7 ===")
    print("1 - Eigene Funktion")
    print("2 - Testfall")

    wahl = input("Auswahl: ") #Eingabe der Auswahl für die Funktion oder den Testfall

    if wahl == "1": #Eingabe der Funktion und des Intervalls sowie der gewünschten Genauigkeit
        formel = input("Funktion: ") #Eingabe der Funktion als String
        a = float(input("a: ")) #Eingabe des Intervallanfangs
        b = float(input("b: ")) #Eingabe des Intervallendes
        epsilon = float(input("epsilon: ")) #Eingabe der gewünschten Genauigkeit

        f = funktion_aus_text_erstellen(formel) #Erstellung der Funktion aus dem eingegebenen String
        x = bisektion_grafik(f, a, b, epsilon) #Berechnung der Nullstelle mit dem Bisektions-Verfahren und Anzeige der Grafiken

        print("Nullstelle:", x) #Ausgabe der gefundenen Nullstelle

    elif wahl == "2": #Testfall für die Funktion x^2 - 25, um die Quadratwurzel von 25 zu finden
        f = funktion_aus_text_erstellen("x**2 - 25")
        x = bisektion_grafik(f, 0, 25, 0.00001)

        print("Nullstelle:", x) #Ausgabe der gefundenen Nullstelle für den Testfall

    else:
        print("Ungültige Auswahl.") #Ausgabe bei ungültiger Auswahl


if __name__ == "__main__":
    plotter()
```

Aufgabe 8 – Polynom

Hier wird ein spezielles Polynom untersucht.

Der Code berechnet die Funktion:
P(x) = 2*x* + *xx* + 3*xxx* - *xxxx*

```
def polynom(x):
    return 2 * x + x**2 + 3 * x**3 - x**4
```

Anschließend wird das Bisektionsverfahren verwendet, um die Nullstelle zu finden.

```
def bisektion_iter(f, a, b, epsilon):
    """ Iterative Version der Bisektion, die die Anzahl der Iterationen zurückgibt
    Args:
        f (function): Funktion, für die die Nullstelle gefunden werden soll
        a (float): Untere Grenze des Intervalls
        b (float): Obere Grenze des Intervalls
        epsilon (float): Gewünschte Genauigkeit"""
    if f(a) * f(b) >= 0:
        print("Kein Vorzeichenwechsel.")
        return None, 0

    iteration = 0 # Zähler für die Iterationen

    while (b - a) / 2 > epsilon: # Solange die Genauigkeit nicht erreicht ist
        c = (a + b) / 2

        if f(a) * f(c) < 0: # Vorzeichenwechsel zwischen a und c
            b = c
        else:
            a = c

        iteration += 1 # Erhöhung der Iterationszähler

    return (a + b) / 2, iteration # Rückgabe der gefundenen Nullstelle und der Anzahl der Iterationen
```

Zusätzlich wird gezählt, wie viele Iterationen benötigt werden.

Der Code führt zwei Tests durch:

- einmal mit niedriger Genauigkeit
- einmal mit hoher Genauigkeit

Am Ende wird erklärt, dass eine höhere Genauigkeit mehr Iterationen benötigt.

Aufgabe 9 – Kettenlinie

Dieser Code löst ein reales Problem: die Berechnung einer durchhängenden Leitung.

Zuerst wird eine Funktion definiert, die den Zusammenhang zwischen dem Krümmungsradius und der Durchbiegung beschreibt. Anschließend wird mit einer zweiten Funktion die Länge der Leitung berechnet.

```
# Aufgabe 9: Krümmungsradius und Länge einer Leitung
def funktion(a):
    return a * math.cosh(50 / a) - a - 10 # Funktion, die den Krümmungsradius a mit der Länge der Leitung in Beziehung setzt

# Berechnung der Länge der Leitung basierend auf dem Krümmungsradius a
def laenge(a):
    return 2 * a * math.sinh(50 / a) #Länge der Leitung basierend auf dem Krümmungsradius a
```

Diese Funktion wird mit dem Bisektionsverfahren und dem Regula Falsi gelöst, um den Krümmungsradius zu bestimmen.

Der Code gibt am Ende aus:

- den Krümmungsradius
- die Länge der Leitung

```
# Ausgabe der Ergebnisse
def ausgabe(name, a):
    """ Ausgabe der Ergebnisse für die Krümmungsradius a und die Länge der Leitung
    Args:
        name (str): Name der Methode (z.B. "Bisektion" oder "Regula Falsi")
        a (float): Gefundener Krümmungsradius a
    """
    if a is not None:
        print(name) #Ausgabe des Namens der Methode
        print("Krümmungsradius a:", a) #Ausgabe des gefundenen Krümmungsradius a
        print("Länge der Leitung:", laenge(a), "m") #Ausgabe der Länge der Leitung basierend auf dem gefundenen Krümmungsradius a
    else:
        print(name, ": Keine Lösung gefunden.")
```